

True if the key-value pair exists and *nil* otherwise. For example:

```
(asoc-contains-pair? alist key value)
;; Syntax

(asoc-contains-pair? '((a . 1) (b . 2)
(c . 3)) 'a 1)
t
(asoc-contains-pair? '((a . 1) (b . 2)
(c . 3)) 'a 2)
nil
```

Access functions

The *asoc-get* function retrieves a key from an *alist*. If the key is not found, *nil* is returned. A couple of examples are shown below:

```
(asoc-get alist key) ;; Syntax

(asoc-get '((b . 2) (a . 1) (e . 5) (d . 4) (c . 3)) 'a)
1
(asoc-get '((b . 2) (a . 1) (e . 5) (d . 4) (c . 3)) 'f)
nil
```

If you want to exclude a list of keys and return an *alist*, you can use the *asoc-dissoc* function. For example:

```
(asoc-dissoc alist keys) ;; Syntax

(asoc-dissoc '((b . 2) (a . 1) (e . 5)
(d . 4) (c . 3)) 'a 'b)
((e . 5) (d . 4) (c . 3))
```

You can add a key to an existing list using the *asoc-put!* function, which accepts an *alist*, a key and a value as arguments. If the key already exists and *replace* is true, then the old entries are replaced. If *replace* is *nil*, then the new entry is added to the front of the *alist*. A few examples are given below:

```
(asoc-put! alist key value &optional
replace) ;; Syntax

(let ((alphabets (list (cons 'a 1)
(cons 'b 2))))
```

```
(asoc-put! alphabets 'c 3)
alphabets)
((c . 3) (a . 1) (b . 2))

(let ((alphabets (list (cons 'a 1)
(cons 'b 2))))
(asoc-put! alphabets 'b 3 nil)
alphabets)
((b . 3) (a . 1) (b . 2))

(let ((alphabets (list (cons 'a 1)
(cons 'b 2))))
(asoc-put! alphabets 'b 3 t)
alphabets)
((b . 3) (a . 1))
```

The *asoc-pop!* function removes a key from an *alist* and returns the same as illustrated below:

```
(asoc-pop! alist key) ;; Syntax

(let ((alphabets (list (cons 'a 1)
(cons 'b 2))))
(asoc-pop! alphabets 'b))
(b . 2)
```

The *asoc-find* API returns the first element in a given *alist* that satisfies the predicate function. For example:

```
(asoc-find predicate alist) ;; Syntax
(asoc-find (lambda (k v) (< v 3)) '((b . 2)
(a . 1) (e . 5) (d . 4) (c . 3)))
(b . 2)
```

The anaphoric variant of the *asoc-find* function is the *asoc--find* function, which is demonstrated below:

```
(asoc--find form alist) ;; Syntax

(asoc--find (lambda (k v) (< v 3)) '((b . 2)
(a . 1) (e . 5) (d . 4) (c . 3)))
(b . 2)
```

The *asoc-find-key* function takes two arguments — a key and an *alist*. It returns the first *alist* element that matches the key, or *nil* otherwise. For example:

```
(asoc-find-key key alist) ;; Syntax
```

```
(asoc-find-key 'c '((b . 2) (a . 1) (e . 5)
(d . 4) (c . 3)))
(c . 3)
(asoc-find-key 'f '((b . 2) (a . 1) (e . 5)
(d . 4) (c . 3)))
nil
```

If you want the list of unique keys in an *alist*, you can use the *asoc-keys* function as illustrated below:

```
(asoc-keys alist) ;; Syntax

(asoc-keys '((b . 2) (a . 1) (e . 5) (d . 4)
(c . 3)))
(b a e d c)
(asoc-keys '((b . 2) (a . 1) (e . 5) (d . 4)
(c . 3) (a . 2) (b . 6)))
(b a e d c)
```

On the other hand, if you want the list of unique values in an *alist*, you can use the *asoc-values* function. A couple of examples are shown below:

```
(asoc-values alist) ;; Syntax

(asoc-values '((b . 2) (a . 1) (e . 5)
(d . 4) (c . 3)))
(2 1 5 4 3)
(asoc-values '((b . 2) (a . 1) (e . 5)
(d . 4) (c . 3) (a . 2) (b . 6)))
(2 1 5 4 3 6)
```

The *asoc-unzip* function performs the reverse operation of *asoc-zip*. It returns all the keys and values from an *alist*. An example is provided below:

```
(asoc-unzip alist) ;; Syntax

(asoc-unzip '((a . 1) (b . 2) (c . 3)))
((a b c) (1 2 3))
```

Loops

The following section addresses a couple of looping constructs possible with *asoc.el*. You can use the *asoc-do* construct to loop through each key-value pair in an *alist* and execute a body of code. In the following example, the sum of all the values is